IT2140- Database Design and Development

Year 2, Semester 1

Lab Sheet 04: Database Connectivity and Data Manipulation in SQL Server

Objective

By the end of this lab session, you will be able to:

- **Connect a JavaFX desktop application with a Microsoft SQL Server database,**
- **Capture user inputs via the form,**
- **Perform basic data manipulation operations including INSERT, UPDATE, and DELETE on the database through the application interface,**
- **Provide user feedback through the application's UI for successful or failed database operations.**

Section 1: Getting Familiar with Microsoft SQL Server and Database Setup

Step 1: Connect to Microsoft SQL Server

- Open Microsoft SQL Server Management Studio (SSMS).
- Connect to your SQL Server instance.

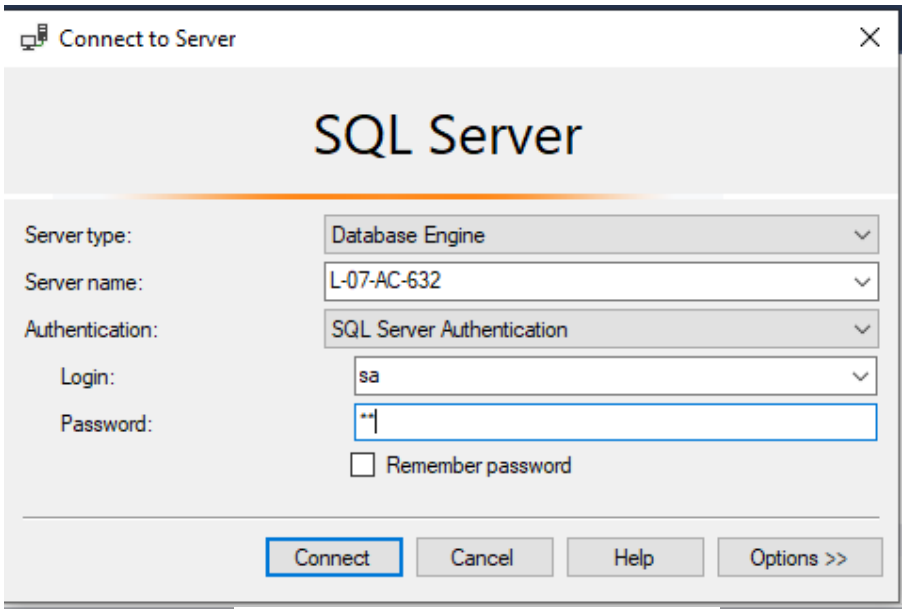


Figure 1: Connect to SQL Server

Step 2: Create a New Database

- Right-click Databases → New Database.
- Name the database *StudentInformation*.

Step 3: Create a Table

- Open a new query window and create the following table by running this SQL command:

```
CREATE TABLE Student (
    ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT,
    GPA FLOAT,
    Address VARCHAR(50)
);
```

Figure 2: Table Query

Section 2: Setting Up Java Project in IntelliJ IDEA

1. Create a new Java Project
2. Add JavaFX and JDBC driver for SQL Server
    - You can download the SQL Server JDBC driver from:

    https://learn.microsoft.com/en-us/sql/connect/jdbc/download-microsoft-jdbc-driver-for-sql-server
3. Add the JAR file to your project via:
    - File → Project Structure → Libraries → + → Java → Add the .jar file

Section 3: Get the JDBC Connection String

You must manually build the JDBC connection string using the following template:

```
jdbc:sqlserver://<server>:<port>;databaseName=<your_db>;encrypt=false;
```

Example (for local server):

```
String url = "jdbc:sqlserver://localhost:1433;databaseName=StudentInformation;encrypt=false;";

String user = "sa";

String password = "your_password";
```

- *localhost: means your SQL Server is running on your own computer.*
- *1433: default port for SQL Server.*
- *encrypt=false: disables SSL if not configured.*

Section 4: Java Code for CRUD Operations

Insert Operation:

Purpose: Adds a new record into the Student table with values entered by the user.

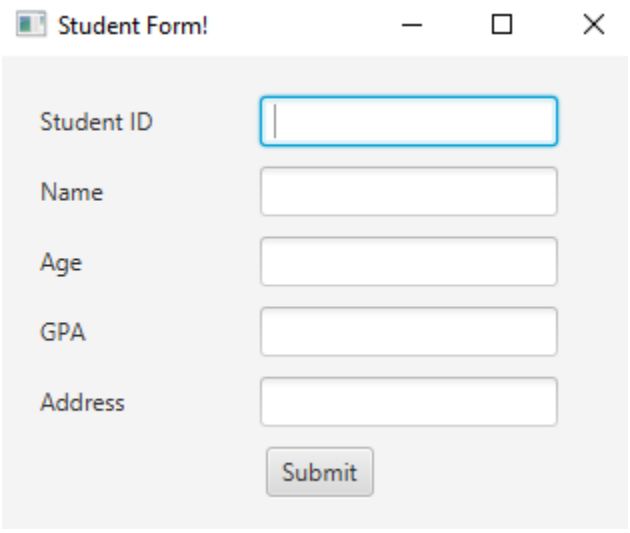


Figure 3: Interface Design

```
String sql = "INSERT INTO Student (ID, Name, Age, GPA, Address) VALUES (?, ?, ?, ?, ?)";

PreparedStatement stmt = conn.prepareStatement(sql);
stmt.setInt(1, Integer.parseInt(txtId.getText()));
stmt.setString(2, txtName.getText());
stmt.setInt(3, Integer.parseInt(txtAge.getText()));
stmt.setDouble(4, Double.parseDouble(txtGpa.getText()));
stmt.setString(5, txtAddress.getText());

stmt.executeUpdate();
```

Update Operation:

Purpose: Modifies an existing student record based on the ID entered.

Update the user interface file and the controller accordingly to support full **CRUD operations.**

```
String sql = "UPDATE Student SET Name = ?, Age = ?, GPA = ?, Address = ? WHERE ID = ?";

PreparedStatement stmt = conn.prepareStatement(sql);
stmt.setString(1, txtName.getText());
stmt.setInt(2, Integer.parseInt(txtAge.getText()));
stmt.setDouble(3, Double.parseDouble(txtGpa.getText()));
stmt.setString(4, txtAddress.getText());
stmt.setInt(5, Integer.parseInt(txtId.getText()));

stmt.executeUpdate();
```

Delete Operation:

Purpose: Removes a student record from the database based on the given ID.

Update the user interface file and the controller accordingly to support full **CRUD operations.**

```
String sql = "DELETE FROM Student WHERE ID = ?";

PreparedStatement stmt = conn.prepareStatement(sql);
stmt.setInt(1, Integer.parseInt(txtId.getText()));

stmt.executeUpdate();
```

Section 5: Run the Application

- Run the Java application via IntelliJ.
- Fill in the form and click Submit.
- View inserted/updated/deleted records directly in SSMS by querying:

```
SELECT * FROM Student;
```